

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: **Operating Systems** **COURSE CODE:** **CSA04**

COURSE OUTCOME COVERED

***C04:** Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks:25

Annexure A

INDUSTRY PROBLEM SOLVING TASK

Code of Conduct:

I ATHIN SIVA.S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Athin Siva S.

1. Large-Scale Storage Company – File Allocation

Problem: A large-scale storage company stores thousands of files with different sizes. The file allocation technique should provide fast access, efficient disk-space utilization, support file growth, and reduce fragmentation. Based on these requirements, Indexed Allocation is a suitable choice.

Suitable File Allocation Method: Indexed Allocation

In indexed allocation, each file is associated with an index block. The index block contains the addresses or pointers of all the disk blocks that contain the actual data of the file. The data blocks do not need to be stored continuously on the disk.

Working

When a file is created, the operating system allocates an index block for that file.

The index block stores pointers to the disk blocks containing the file data.

The data blocks can be located at different positions on the disk.

When a user requests a particular part of the file, the operating system uses the index to locate the required block.

For very large files, multi-level indexing can be used to store a larger number of block addresses.

Advantages

Provides efficient direct or random access to file blocks.

Avoids the external fragmentation associated with contiguous allocation.

Allows files to grow dynamically without requiring adjacent free blocks.

Suitable for files of different sizes, including large files.

Provides flexible disk-space management.

Conclusion

Indexed allocation is a suitable method for a large-scale storage company because it combines flexible storage with efficient random access. It is especially useful when files have different sizes and may increase in size over time.

2. Data Center – Disk Scheduling

Problem: A data center may receive a very large number of disk I/O requests simultaneously. If requests are processed inefficiently, the disk head may travel unnecessarily, increasing seek time and reducing system performance. Therefore, a suitable disk scheduling algorithm is required.

Recommended Algorithm: SCAN

The SCAN disk scheduling algorithm, also known as the Elevator Algorithm, is suitable for a busy data center. The disk head moves in one direction and services all pending requests along its path. When it reaches the end of the disk or the last required request, it reverses direction and services requests on the return path.

Working

The operating system maintains a queue of pending disk requests.

The disk head starts from its current position and moves in a selected direction.

While moving, it services all requests encountered along the path.

After reaching the end of the disk or the last request in that direction, it reverses direction.

The process continues as new requests arrive.

Advantages

Reduces unnecessary disk-head movement compared with FCFS.

Provides better average seek performance for heavy workloads.

Provides fairer service than SSTF and reduces the possibility of starvation.

Works well when a large number of disk requests are waiting.

Provides more predictable performance than simply servicing requests in arrival order.

Conclusion

SCAN is a good choice for a busy data center because it balances performance and fairness.

It reduces excessive head movement while ensuring that requests at different disk locations eventually receive service.

3. Bitmap-Based Free Space Management

Problem: An operating system uses a bitmap to identify free and allocated disk blocks. In a large storage system, searching the bitmap bit by bit can become slow, particularly when the system needs to locate a large number of consecutive free blocks.

Optimization Technique: Hierarchical Bitmap with Word-Level Searching

A better solution is to combine a hierarchical bitmap with word-level bit operations.

Instead of checking every individual bit, the operating system can divide the bitmap into groups and use a higher-level bitmap to identify which groups contain free blocks.

Working

A bit value of 1 can represent an allocated block, while 0 represents a free block.

The bitmap is divided into groups or machine words.

A higher-level bitmap records whether each group contains free blocks.

The operating system first searches the higher-level bitmap to find a suitable group.

It then performs word-level or processor-supported bit operations to locate the exact free block or sequence of blocks.

Benefits

Reduces the number of bitmap entries that must be examined.

Improves the speed of free-space allocation.

Reduces CPU overhead during file creation and file expansion.

Works efficiently with very large disks.

Can locate contiguous free blocks faster than a simple linear bitmap scan.

Conclusion

A hierarchical bitmap combined with word-level bit searching is an efficient optimization for large-scale operating systems. It reduces search time and improves the overall performance of disk-space allocation.

4. Real-Time Application – SCAN vs C-SCAN

Problem: A real-time application requires consistent and predictable disk access times. In such systems, an algorithm should not give significantly different waiting times to requests depending only on their physical position on the disk. Therefore, SCAN and C-SCAN must be compared.

SCAN

SCAN services requests while the disk head moves in one direction and then reverses direction. It provides good fairness, but requests near one end of the disk may experience different waiting times compared with requests at other locations.

C-SCAN

C-SCAN, or Circular SCAN, services requests in only one direction. When the disk head reaches the end of the disk, it returns to the beginning without servicing requests during the return movement. It then continues servicing requests in the same direction.

Why C-SCAN is More Suitable

Provides a more uniform waiting time for requests.

Reduces the bias that can occur in SCAN because the head repeatedly reverses direction.

Provides predictable service for requests distributed across the disk.

Is useful when consistent response time is more important than minimizing every individual seek.

Provides good fairness under a heavy disk workload.

Conclusion

For a real-time application where predictable disk access is a major requirement, C-SCAN is generally more suitable than SCAN. Its one-direction servicing pattern provides more uniform waiting times and better fairness across the disk.

5. Hybrid File Allocation and Disk Scheduling

Problem: A large-scale data access system needs an efficient file allocation technique as well as an efficient disk scheduling algorithm. The system must handle large files, support random access, use disk space efficiently, and provide predictable disk-request servicing.

Proposed Hybrid Solution: Indexed Allocation + C-SCAN

1. Indexed Allocation

Indexed allocation is used at the file-system level. Each file has an index block containing pointers to the data blocks. Since the data blocks can be located at different disk locations, the method is flexible and avoids the need for contiguous free space.

Supports direct and random access.

Allows large files to grow dynamically.

Reduces external fragmentation.

Works efficiently when file blocks are distributed across the disk.

2. C-SCAN Disk Scheduling

C-SCAN is used to manage the disk I/O request queue. The disk head services requests in one direction. After reaching the end, it returns to the beginning and again services requests in the same direction.

Provides more uniform waiting times.

Reduces starvation and improves fairness.

Works well for systems with heavy I/O workloads.

Provides predictable disk-request handling.

Why Combine Both Methods?

System Requirement	Indexed Allocation	C-SCAN
Large file support	✓	—
Random access	✓	—
Reduced external fragmentation	✓	—
Efficient disk request handling	—	✓
Predictable waiting time	—	✓
Large-scale storage system	✓	✓

Overall Working

The application first accesses files through the file system using indexed allocation. The resulting disk I/O requests are placed in the request queue. The C-SCAN scheduler then determines the order in which these requests are serviced by the disk.

Application → File System → Indexed Allocation → Disk I/O Queue → C-SCAN Scheduler → Disk

Conclusion

The combination of Indexed Allocation and C-SCAN provides a balanced solution for large-scale data access. Indexed allocation manages large and dynamically growing files efficiently, while C-SCAN provides fair and predictable disk scheduling. Together, they

improve storage flexibility, disk utilization, access performance, and overall system reliability

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

		evaluation			
--	--	------------	--	--	--